

Apunte de la unidad 1: Objetos

Programación con Objetos 1



Unidad 1: Objetos

1.1 Paradigmas de programación

1.1.1 Ejemplo: Pepita

1.2 Objetos, mensajes y métodos

Referencias y Estado Interno

Acciones y consultas

1.3 Breve introducción a WolloK

1.3.1 Instalación y creación de un proyecto ejemplo

1.3.2 Definición de objetos

1.3.2.1 REPL

1.3.2.2 Tipado implícito

1.3.3 Objetos básicos

Números

Números fraccionarios

Booleanos

Cadenas (Strings)

Fechas

1.3.5 Objetos que colaboran

1.3.6 Métodos accesors

1.3.8 Variables vs constantes

ahora 1.4 Ambiente y referencias

1.4.1 Referencia a sí mismo: self

1.4.2 Objetos autodefinidos (WKO) vs Referencias globales

1.5 Introducción al polimorfismo

1.5.1 Ejemplo de polimorfismo

1.5.2 Mensajes, métodos y method lookup

1.5.3 Polimorfismo y Tipos

1.6 Malas prácticas

[1.6.1 Mala práctica: if que roba responsabilidad](#)

[1.6.2 Mala práctica: distintos mensajes para la misma responsabilidad](#)

[1.6.3 Mala práctica: usar referencias globales en los métodos](#)

[1.6.4 Mala práctica: utilizar atributos cuando se necesitan parámetros](#)

[1.6.5 Mala práctica: utilizar parámetros cuando se necesitan parámetros](#)

Unidad 1: Objetos

1.1 Paradigmas de programación

Un **paradigma** define una visión sobre un campo de estudio en particular. Más específicamente a la programación, los distintos paradigmas establecen formas de analizar y resolver los problemas de ese campo de estudio. Cada paradigma aplica/define/utiliza un conjunto de herramientas para resolver problemas sobre ese campo.

Cuando se refiere a los paradigmas de programación, en primer lugar se debe delimitar los problemas computacionales. Un problema computacional es aquel que se resuelve mediante la ejecución de un programa. Si bien muchos de los problemas a los que las personas se enfrentan diariamente no son computacionales, frecuentemente es posible encontrar un problema computacional asociado:

Problema NO computacional	Problema computacional
Ordenar personas en una fila	Ordenar números
Que una persona curse una materia en la universidad	Inscribir una persona en una materia de la universidad
Cargar suficiente combustible para llegar a destino	Simular el consumo de energía de un vehículo

La tarea de construcción de un programa puede definirse como encontrar las construcciones específicas que ofrece el paradigma (también llamadas abstracciones) para permitir la resolución del problema en cuestión. La forma en que se pueden/deben organizar las abstracciones está determinada por el paradigma específico de programación.

Entre los paradigmas imperativos se encuentran el paradigma estructurado y el paradigma orientado a objetos, mientras que los paradigmas denotacionales o declarativos incluyen el paradigma funcional y el paradigma lógico.

1.1.1 Ejemplo: Pepita

El problema es el siguiente: un ornitólogo estudia a la golondrina pepita. Él es capaz de observar las distancias que vuela y los instantes en que descansa. Sabe que este comportamiento altera su energía interna, haciendo que Pepita esté cansada o no. Saber si la golondrina Pepita está cansada o no el día de hoy es el problema no computacional de la vida real.

Él quiere un sistema que le dé apoyo para realizar este cálculo. Él es un experto que conoce las reglas de negocio. Sabe que:

- Pepita amanece con una energía de 100 calorías
- Cuando vuela, consume diez calorías para despegar, más una caloría por cada diez metros recorridos.

- Cuando descansa recupera 10 calorías
- Está cansada si su energía es menor a 30 calorías

Con esa información se puede generar un modelo computacional (resuelve un problema computacional) que modele a la golondrina Pepita y realice los cálculos que corresponden a lo que el ornitólogo observa durante un día, permitiendo que el ornitólogo solo ingrese la actividad de pepita y consulte cuando lo desee si pepita está cansada o no.

1.2 Objetos, mensajes y métodos

Los objetos son las abstracciones básicas de un programa orientado a objetos. Un programa construido en el paradigma Orientado a Objetos es, básicamente, una colección de objetos. Cada objeto modela una idea interesante para resolver un problema. Como primera aproximación, se puede pensar que cada concepto interesante del problema real que se quiere resolver es un objeto que exhibe un comportamiento. Entonces, los objetos reciben mensajes para activar tales comportamientos. Estos mensajes pueden estar originados en el mismo objeto, en otros objetos o a partir de una acción de la persona que usa el sistema.

Una definición relacionada a la de un sistema (es decir *conjunto de partes que se relacionan para un objetivo común*), es la siguiente:

“Un sistema es un conjunto de objetos
que se envían mensajes para alcanzar un determinado objetivo”

Pepita es una idea importante para nuestro problema real; es decir, al pasar esta idea a nuestro modelo computacional, se modela como un objeto. Al analizar el problema, se observa que su comportamiento consiste en volar, descansar y saber si está cansada o no. Cada uno de estos comportamientos estará detrás de un mensaje: *volar*, *descansar*, *cansada*. El conjunto de mensajes que entiende un objeto constituye su **interfaz**.

En el objeto Pepita, se debe definir un **método** (comportamiento) para cada mensaje. En el método se escribe el código que se ejecuta cuando el objeto recibe el mensaje.

Los objetos interactúan entre sí mediante mensajes. Y para que un objeto envíe un mensaje a otro, es necesario que primero lo conozca mediante una referencia.

Referencias y Estado Interno

Se ha mencionado que un objeto puede recibir mensajes de otros objetos o de acciones que realiza quien usa el sistema. Para que eso suceda, es necesario contar con una **referencia** que *apunte* al objeto. Nunca se habla directamente con un objeto: siempre es a través de una referencia. A diferencia del paradigma estructurado, en el que cada variable o parámetro tiene un valor, en el paradigma de objetos estos conceptos son referencias que apuntan a un objeto. Si bien coloquialmente se dice que un objeto se pasa por parámetro, a

veces es importante entender que, técnicamente, se pasa una referencia al objeto por parámetro.

Hay tres situaciones en las que un objeto puede trabajar con una referencia a otro objeto:

- La recibió por parámetro.
- Es una **referencia global**, y entonces puede ser usada desde cualquier lado del sistema
- Es parte de su **estado interno**: un objeto puede definir un conjunto de referencias que sólo él conoce. A cada una de estas referencias se la denomina **atributo** (en alguna bibliografía se la llama variable de instancia)

El conjunto de referencias que tiene un objeto se denomina **estado interno**. Además de referencias a otros objetos definidos a partir del problema a resolver (dominio), es posible tener referencias a **objetos nativos** del lenguaje que representan valores literales como números o *strings* de texto.

Considerar el escenario de la figura 1, donde se tienen 2 objetos (en color azul) definidos por quien programa y 2 objetos nativos (en color verde) que representan un valor entero y un booleano respectivamente. El estado interno de Ada está compuesto por los atributos *mascota* y *edad*, mientras que el estado interno de Michi tiene el atributo *dormida*.

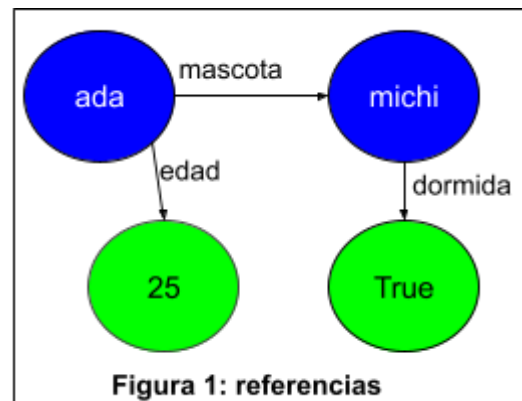


Figura 1: referencias

Al modelar una solución con objetos, procuraremos que el estado de un objeto sea manipulado únicamente por los métodos definidos en el propio objeto. Cada referencia define un **atributo** con un nombre y un valor: el objeto al que *apunta*. El nombre de un atributo debe elegirse para describir el rol que cumplirá el objeto referido para el dueño del atributo, y no tanto con el objeto referido en sí. En el ejemplo de la figura, Ada no sabe que su mascota es Michi, sino que sabe que tiene una mascota a la que enviar mensajes. De la misma manera, Ada sabe que tiene una edad, no le interesa en sí que sea el número 25. Con el andar del programa, la referencia a la edad podría dejar de apuntar al 25 para pasar a referirse a otro objeto numérico.

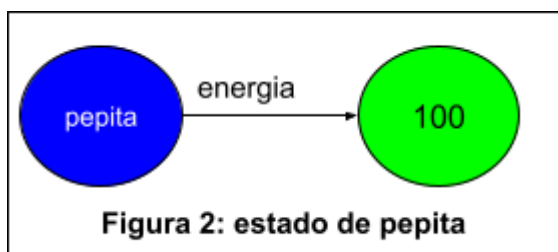


Figura 2: estado de pepita

Centrándonos en el ejemplo del ornitólogo, hemos definido hasta aquí 3 mensajes: **volar**, **descansar** y **cansada**. Para cada uno de estos mensajes se define un método con el código que se ejecutará al recibirlo. Cada uno de estos mensajes requiere trabajar con *la energía de Pepita*. Esta es una información que depende

de Pepita y se debe mantener entre los distintos envíos de mensajes. Por esa razón es que la manera de modelar la energía en Pepita es a través de un atributo: una referencia interna que comenzará apuntando a 100 (porque así nos exige el dominio) e irá cambiando el número apuntado según Pepita reciba los mensajes volar y descansar.

Acciones y consultas

Cada mensaje tiene asociada una **responsabilidad** (el qué se debe hacer), conocida tanto por quien lo envía como por el objeto receptor. Mientras que la manera en la que el método cumple dicha responsabilidad (el cómo se realiza) queda encapsulada en el receptor y no es visible para el emisor del mensaje.

El mensaje `cansada` provoca una respuesta por parte del objeto, obteniéndose un valor que luego podría utilizarse como tal en otros mensajes. Es una **consulta** que devuelve un objeto, en este caso, booleano (verdadero o falso). La manera de decidir la respuesta en el método (evaluando las calorías que tiene según su atributo `energía`) es una cuestión interna al objeto y no le interesa a quien envía el mensaje.

El mensaje `volar` se sabe que es una **acción u orden**. Esto no genera una respuesta directa, ya que no se trata de una consulta. La responsabilidad es indicar que se produjo un vuelo. El emisor sabe que el método aplicará un **efecto** en el estado interno del objeto (efectivamente, Pepita internamente disminuirá su energía), pero no sabe cuál es.

El mensaje `descansar`, dualmente al anterior, es un mensaje de tipo orden, con un efecto contrario: lo que haga internamente debe aportar energía. Eventualmente podría hacer que Pepita ya no esté cansada.

Según las responsabilidades de los mensajes, podemos agruparlos en dos grandes grupos:

Consulta: mensaje que no tiene efecto en el estado interno y tiene un respuesta

Orden: mensaje que generalmente tiene efecto en el estado interno y nunca tiene respuesta

En algunos problemas avanzados se pueden generar mensajes que tengan efecto y también respuesta. Pero esto se reserva para ciertos tipos de problemas especiales bien estudiados. En una introducción a la programación con objetos, se prefiere que los mensajes correspondan a alguna de estas dos categorías de manera excluyente: es una consulta o es una orden.

Como las consultas no tienen efecto, pueden invocarse varias veces sin alterar el estado del objeto, mientras que las órdenes se deben invocar sólo cuando es necesario generar un efecto. Consecuentemente, es importante asegurar que no se invoquen órdenes desde ninguna consulta porque eso haría que la consulta tuviera efecto.

1.3 Breve introducción a Wollok

Wollok es un lenguaje de programación orientado a objetos definido con fines didácticos, es decir, que se enfoca en los conceptos que se quieren ejercitar y para eso recorta aquellos elementos que distraen a la persona aprendiz de su objetivo. En particular, se integra a un entorno de programación (IDE) como *Eclipse* o *Visual Studio Code*.

1.3.1 Instalación y creación de un proyecto ejemplo

Para su integración con Visual Studio Code (VSC) es necesario:

1. Instalar VSC: podés seguir [estos pasos](#)
2. Instalar la extensión de Wollok siguiendo [estos pasos](#)

Una vez instalado, es posible crear un proyecto de ejemplo que se puede usar como molde o *template* para nuevos proyectos, siguiendo [este tutorial](#) de la pagina de wollok.

1.3.2 Definición de objetos

La sintaxis para definir un objeto en Wollok tiene la siguiente estructura:

```
object nombreDelObjeto {  
  <atributos>  
  <comportamiento>  
}
```

En el ejemplo definimos un objeto **pepita** que tiene una cantidad de energía que le permite volar, y que puede descansar para reponer energía. Más detalladamente, al volar de **pepita** disminuye su energía en 10 unidades y una caloría cada 10 km. Al descansar aumenta su energía en 10 unidades. Además, el valor inicial del atributo energía es 100.

```
object pepita {  
  var energia = 100  
  
  method volar(distancia) {  
    energia = energia - 10 - distancia/10  
  }  
  
  method descansar() {  
    energia = energia + 10  
  }  
  
  method cansada() {  
    return energia < 30  
  }  
}
```

Es importante observar que los mensajes `volar(distancia)` y `descansar()` al ser órdenes no devuelven nada y alteran el estado interno haciendo que la referencia energía cambie el objeto numérico al que apunta, mientras que `cansada()` es una consulta y por lo tanto tiene un `return` para indicar que es lo que devuelve.

1.3.2.1 REPL

Una vez programado el objeto en un archivo `wlk` (`example.wlk` es por defecto). Aparecerá sobre el objeto un link con el nombre *run in REPL*. Esto abre una pequeña consola a partir de la cual se pueden enviar mensajes a nuestros objetos utilizando una referencia global que se llama igual que nuestro objeto y utilizando la notación de *objeto.mensaje(parametros)*

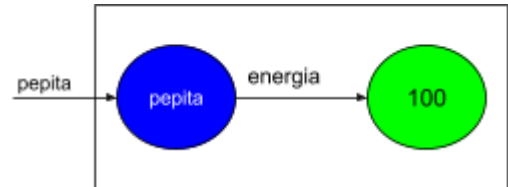


Figura 2: referencia global de pepita

```

wollok:example> pepita.cansada()
✓ false
wollok:example> pepita.volar(601)
✓
wollok:example> pepita.cansada()
✓ true
wollok:example> pepita.descansar()
✓
wollok:example> pepita.cansada()
✓ false
  
```

1.3.2.2 Tipado implícito

La definición de un atributo o de un parámetro se realiza indicando el nombre (y nada más), lo que establece una nueva referencia que estará disponible en el ámbito correspondiente. Si es un atributo, podrá utilizarse desde cualquier método del objeto, mientras que si es un parámetro, sólo podrá utilizarse en el método definido.

Dado que el nombre no garantiza un tipo o característica alguna al respecto de ese parámetro, no puede asegurarse de que, en el momento de la ejecución del método, este haga referencia a un número, a una fecha o a un objeto con determinada interfaz. Es responsabilidad de quien programa enviar los mensajes adecuados. En caso de que un objeto referenciado reciba un mensaje que no entiende se lanzará un error.

1.3.3 Objetos básicos

La distribución de Wollok aporta un conjunto de objetos básicos (nativos) que resultarán muy útiles para poder construir los objetos del dominio específico.

Números

Los números en Wollok se representan como objetos inmutables, esto quiere decir que, por un lado, un número no cambia su estado interno y por el otro, que la suma de dos objetos número resulta en un nuevo objeto número.


```
const a = 1
var b = a + 10 // suma
b = b - 1      // resta
b = b * 2      // multiplicación
b = b / 2      // división
b = b % 2      // resto
b = b ** 3     // elevado a (3 en este caso)
5.between(2, 7) // preguntamos si 5 está entre 2 y 7 ==> sí
3.min(6)       // el menor número entre 3 y 6 ==> 3
3.max(6)       // el mayor número entre 3 y 6 ==> 6
(3.1416).truncate(0) // la parte entera de 3.1416 ==> 3 --
(3.1416).truncate(2) // 3.14
(3.1416).roundUp(0)  // el primer entero mayor a 3.1416 ==> 4 --
(3.1416).roundUp(2)  // 3.15
```

Números fraccionarios

Por defecto, WolloK usa un formato de números fraccionarios con a lo sumo 5 dígitos decimales y redondea hacia arriba si el último decimal es 5 o mayor; si no, hacia abajo. Además, para imprimir los números elimina los ceros no representativos.

```
> 5.2912413
✓ 5.29124
> 5.2912488
✓ 5.29125
> 1.000005 - 1.000004
✓ 0.00001
> 1.000002 - 1.000001
✓ 0
```

Booleanos

Hay dos objetos booleanos representados con los literales “true” y “false”. Al igual que los números también son objetos inmutables, es decir que una expresión de verdad como por ejemplo (true ^ false) no modifica el objeto original.

```
const hecho = true and true
const esTrue = true
const esFalse = false
const seraFalse = esTrue and esFalse
const seraTrue = esTrue or esFalse
const seraTrue = not false
```

Además, WolloK dispone de otra sintaxis para los operadores booleanos, que resulta cómoda para quienes programan en otros lenguajes como C o Java, utilizando caracteres no alfabéticos:

- and: `a && b`
- or: `a || b`
- not: `!a`

Cadenas (Strings)

Las cadenas de caracteres se delimitan con una o dos comillas.

```
const unString = "hola"  
const otroString = 'mundo'
```

También son objetos inmutables, es decir que al concatenar dos cadenas se obtiene un nuevo objeto de tipo cadena. Por ejemplo, concatenando “hola” y “mundo” mediante el operador + se tiene el nuevo objeto String, como se ve a continuación:

```
const holaMundo = unString + " " + otroString + " !"
```

Fechas

Una fecha es un objeto inmutable que representa un día, mes y año (sin horas ni minutos). Se crean de dos maneras posibles:

```
> const hoy = new Date()  
    // toma la fecha del día  
> const unDiaCualquiera = new Date(day = 24, month = 11, year = 2017)  
    // se ingresa en formato día, mes y año
```

Algunas operaciones que podemos hacer con las fechas son:

```
> const hoy = new Date()  
✓  
> hoy  
✓ 24/11/2017  
> hoy.plusYears(1)    // sumo un año  
✓ 24/11/2018          // devuelve una nueva fecha  
> hoy.plusMonths(2)   // sumo 2 meses  
✓ 24/1/2018  
> hoy.plusDays(20)  
✓ 14/12/2017  
> hoy.isLeapYear()    // pregunto si el año es bisiesto
```

```
✓ false
> hoy.dayOfWeek()      // qué día de la semana es
✓ "friday"
> hoy.month()
✓ 11
> hoy.year()
✓ 2017
> const ayer = hoy.minusDays(1)
✓                               // resto un día para obtener el día de ayer
> ayer < hoy            // comparo fechas
✓ true
> ayer - hoy           // comparo fechas
✓ -1                     // diferencia en días entre ayer y hoy
> const haceUnMes = hoy.minusMonths(1)
✓
> ayer.between(haceUnMes, hoy)
✓ true                   // ayer está entre hace un mes y hoy
```

1.3.5 Objetos que colaboran

Al comienzo de este apunte se definió un sistema como un conjunto de objetos que colaboran para resolver un problema (modelar una situación), y para que un objeto pueda enviarle un mensaje a otro objeto, necesita tener su referencia.

Se amplía el ejemplo de pepita con esta definición: *cuando pepita come alpiste aumenta 20 calorías de energía.*

En este escenario el alimento está modelado por un objeto **alpiste**, de manera que cuando pepita come alpiste, éste le aporta una cierta cantidad de energía. Determinar qué cantidad específica aporta es responsabilidad del objeto **alpiste**.

Para modelar esta situación, se necesita que Pepita entienda el mensaje **comer(alimento)** como se ve a continuación. Además, se necesita que el objeto alpiste entienda el mensaje **energíaQueAporta()**, que es una **consulta**, pues retorna un valor (en este caso numérico).

```
object pepita {
  ...
  method comer(alimento) {
    energia = energia + alimento.energíaQueAporta()
  }
}
```

```
object alpiste {  
  method energiaQueAporta() {  
    return 20  
  }  
}
```

1.3.6 Métodos *accessors*

En ciertas ocasiones, el problema en cuestión nos exige que cierto estado interno sea expuesto por fuera del objeto. Con el enunciado actual, el atributo energía de Pepita está encapsulado completamente, pero suponga la siguiente extensión al problema:

- El ornitólogo no sólo quiere saber si Pepita está cansada o no, también quiere saber cuál es la energía de pepita.
- El ornitólogo quiere poder configurar la energía de pepita dando un valor distinto al inicial

El estado interno de un objeto no puede ser accedido directamente por fuera del objeto. El objeto debe ser programado con mensajes que permitan exponerlo. Este tipo de problemas es tan común que los mensajes/métodos que lo resuelven tienen nombres especiales llamados **accessors**, cuyo fin es publicar el objeto referenciado por un atributo (**getter**) o modificar el estado interno (**setter**). Cada tecnología suele tener su propia convención para éstos métodos.

Continuando el ejemplo de **pepita** agregamos un método getter y uno setter para el atributo energía:

```
object pepita {  
  var energia = 100  
  ...  
  method energia() {  
    return energia  
  }  
  method energia(_energia) {  
    energia = _energia  
  }  
}
```

El método getter es un método de una sola instrucción de tipo return, que devuelve el atributo (la referencia) con el mismo nombre (en este caso, energía). El método setter recibe un parámetro que, siguiendo la convención propuesta por Wolok, se nombra como el atributo usando un “_” como prefijo.

1.3.8 Variables vs constantes

Wollok tiene dos maneras de definir referencias, considerando si pueden cambiar o no. En el primer caso, cuando el objeto al que se hace referencia mediante un nombre puede cambiar, se utiliza el concepto de variable, denotado con la palabra **var**. En el segundo caso, cuando la referencia es fija o no puede cambiar una vez inicializada (no es válida una nueva operación de asignación), se considera constante y se denota con la palabra **const**.

<pre>var ave = pepita ave = pepon</pre>	<pre>const edadAdulta = 21 edadAdulta = 18 // ESTO DA ERROR</pre>
Referencia variable	Referencia constante

1.4 Ambiente y referencias

En este paradigma los objetos necesitan un espacio donde convivir y comunicarse, denominado **ambiente**. En otras tecnologías, este concepto se conoce como imagen.

Por otro lado, como mencionamos en el apartado anterior, para que los objetos puedan comunicarse entre sí en el ambiente, es necesario que se vinculen mediante referencias. Esta vinculación puede darse de diferentes formas; algunas de ellas son: en el alcance de un método o como parte del estado interno de otro objeto.

La primera situación se puede ejemplificar a través del método `comer(alimento)`, en cuyo ámbito es posible identificar dos referencias al mismo objeto `alpiste`: la que ya se tenía del objeto en el ambiente de ejecución (color verde) y el nombre del parámetro (color rojo).



Figura 2: referencias dentro del método comer()

Para visualizar la segunda situación, considerar que se define el objeto `rebeca` que cuida de `pepita`, entrenándola, haciendo que descansa y dándole de comer. Para enviarle los mensajes correspondientes, primero necesita tener la referencia al objeto `pepita` como parte de su estado interno. Esto se describe a continuación:

```
object rebeca {
  var ave
  method ave(_ave) {
```

```

    ave = _ave
  }
}

```

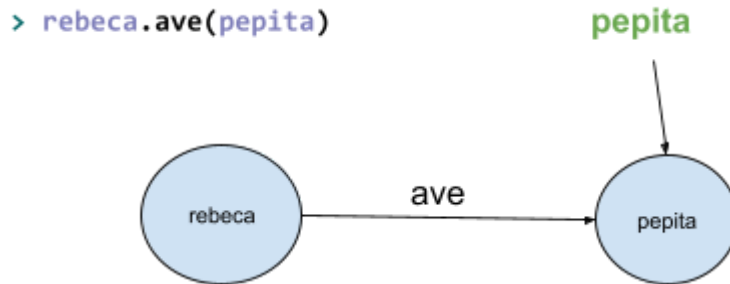


Figura 3: objeto como estado interno de otro objeto

1.4.1 Referencia a sí mismo: self

En ocasiones es útil descomponer el comportamiento asociado a un mensaje en varios métodos del mismo objeto. Estas situaciones generan la necesidad de tener un recurso del lenguaje que permita a un objeto enviarse mensajes a si mismo. Wolok provee la palabra reservada **self** que es una referencia al propio objeto, de la que pueden disponer todos los objetos.

Otra situación es cuando necesitamos hacer referencia al mismo objeto en el que estamos escribiendo el mensaje.

Considerar los siguientes ejemplos:

- Se quiere abstraer el cálculo de la energía que se gasta al volar en un método de consulta
- Se quiere mantener una relación bidireccional entre Rebeca (la entrenadora de Pepita) y el ave que entrena, de tal manera que Rebeca conoce al ave entrenada y Pepita conoce a su entrenadora. Este tipo de problemas que tienen referencias cruzadas hay que tratarlos con cuidado para no entrar en ciclos infinitos. La inicialización de un atributo apuntando a *null* indica que la referencia no tiene un objeto aún al cual apuntar y deberá ser asignada en otro momento.

```
object rebeca {
  var ave = null
  method ave(_ave) {
    ave = _ave
    _ave.entrenador(self) //se usa self para pasarse como argumento
  }
}

object pepita {
  var energia = 100
  var entrenador = null

  method entrenador(_entrenador) {
    entrenador = _entrenador
  }

  method volar(distancia) {
    //se usa self para enviarse un mensaje a si misma
    energia = energia - self.energiaQueGasta(distancia)
  }

  method energiaQueGasta(distancia) {
    return 10 + distancia/10
  }
}
```

1.4.2 Objetos autodefinidos (WKO) vs Referencias globales

Los objetos, como se definieron hasta ahora, son *objetos autodefinidos* o *well-known objects* (wko) que dejan disponible en el ambiente una referencia para poder enviarles mensajes desde la consola. Esto permite que cualquier otro objeto pueda enviarle un mensaje sin necesidad de guardar una referencia propia, como se ve en el código que sigue, donde el objeto *juan* no tiene una referencia (var o const) al objeto *firulais*:

```
object firulais {
  method jugar() {
```

```
//  
}  
}  
object juan {  
  method jugarConMascota(){  
    firulais.jugar()  
  }  
}
```

La anterior no es una buena práctica porque establece que el objeto *Juan* solo puede enviar ese mensaje a *Firulais* (no puede tener otra mascota). Si bien puede ser tentador ahorrar trabajo utilizando los objetos autodefinidos desde las referencias globales, al hacerlo se está perdiendo la posibilidad de trabajar con el rol de "mascota" que está cumpliendo *firulais* para *juan*. Tener un atributo *mascota* que apunte a *firulais* da la posibilidad de que otros objetos puedan cumplir con ese rol, tal cual se trabaja en la sección siguiente.

1.5 Introducción al polimorfismo

Cada mensaje declarado por un objeto especifica el nombre de la operación, los objetos que toma como parámetros. Esto es lo que se conoce como la firma (*signature* en inglés) del mensaje. Al conjunto de todas las firmas definidas por las operaciones de un objeto se le denomina la **interfaz del objeto**. Dicha interfaz caracteriza el conjunto completo de mensajes que se pueden enviar al objeto: cualquier mensaje que concuerde con una firma de la interfaz.

Las interfaces son fundamentales en los sistemas orientados a objetos. Los objetos sólo se conocen a través de su interfaz. No hay modo de saber nada de un objeto o pedirle que haga nada si no es a través de su interfaz. La interfaz de un objeto no dice nada acerca de su implementación y, por lo tanto, distintos objetos son libres de resolver los mensajes de forma diferente. Eso también implica que dos objetos con métodos completamente diferentes pueden tener interfaces idénticas. Es decir, los dos objetos entienden los mismos mensajes, porque tienen métodos con igual firma (nombre y parámetros), pero sus implementaciones (el código) son distintas.

Cuando se envía un mensaje a un objeto, la operación concreta que se ejecuta depende tanto del mensaje como del objeto que la recibe, pues objetos diferentes que soportan mensajes idénticos pueden tener distintos métodos que satisfacen esos mensajes. La asociación en tiempo de ejecución entre un mensaje a un objeto y el método que se ejecuta es lo que se conoce como **enlace dinámico**. El enlace dinámico significa que enviar un mensaje no nos liga a un código particular hasta el momento de la ejecución. Por tanto, es posible escribir programas que esperen un objeto con una determinada interfaz, sabiendo que cualquier objeto que tenga la interfaz correcta aceptará el mensaje. Más aún, el enlace dinámico nos permite sustituir objetos en tiempo de ejecución por otros que tengan la misma interfaz. Esta capacidad de sustitución es lo que se conoce como **polimorfismo**, y es un concepto clave en los sistemas orientados a objetos. Permite que un objeto solicitante (emisor) haga pocas suposiciones sobre otros objetos aparte de que permitan una interfaz determinada. El polimorfismo simplifica las definiciones de los solicitantes, *desacopla* unos objetos de otros y permite que varíen las relaciones entre ellos en tiempo de ejecución.

1.5.1 Ejemplo de polimorfismo

En el ejemplo de pepita, sabemos que puede comer alpiste:

```
object pepita {  
  var energia = 100  
  
  method comer(alimento) {  
    energia = energia + alimento.energiaQueAporta()  
  }  
}  
  
object alpiste {  
  method energiaQueAporta() {  
    return 20  
  }  
}
```

Se agrega el siguiente requerimiento:

- Pepita puede alimentarse tanto con alpiste (que aporta 20 calorías de energía) o con una manzana. La energía que aporta una manzana es una base de 5 multiplicado por un grado de madurez. la madurez comienza en 1 y puede ir incrementándose a medida que pasa el tiempo.

Para resolver el nuevo requerimiento, es importante entender que no es necesario modificar el objeto pepita. La clave está en ver que la relación entre pepita y alpiste no está dada por su identidad: no importa cual es el alimento que se le pasa a pepita, lo que importa es que para pepita un alimento es cualquier cosa que entienda el mensaje *energiaQueAporta()*

La solución es tan simple como programar un objeto manzana que implemente dicho método

```
object manzana {  
  const base = 5  
  var madurez = 1  
  method madurar() {  
    madurez = madurez + 1  
  }  
  method energiaQueAporta() {  
    return base * madurez  
  }  
}
```

Un ejemplo de uso en el REPL:

```
wollok:example> pepita.comer(alpiste)
✓
wollok:example> pepita.energia()
✓ 120
wollok:example> pepita.comer(manzana)
✓
wollok:example> pepita.energia()
✓ 125
```

Cuestiones importantes para tener en cuenta:

- Aparece el concepto de "*alimento*" como una interfaz: Cualquier cosa que entienda el mensaje *energiaQueAporta* es un alimento para *pepita*.
- Se dice que *manzana* y *alpiste* son objetos **polimórficos** para *pepita*, ya que pueden ser usados indistintamente por *pepita*
- *pepita* en ningún momento sabe (ni debe saber) si está comiendo alpiste o manzana
- El hecho de que sea un único mensaje el que ambos alimentos tienen que implementar se debe a este caso concreto, podría darse que *pepita* necesite enviar dos mensajes distintos a los alimentos. En ese caso los alimentos tendrían que implementar ambos mensajes
- No es necesario que *alpiste* y *manzana* compartan su interfaz completamente, solo los mensajes que tienen que ver con el concepto alimento deben ser implementados. Por ejemplo, *manzana* tiene un método *madurar* que no existe en *alpiste*.
- Al ver el método alimentar de *pepita* no se puede identificar si va a aumentar 20 (por alpiste) o 5 (por la manzana). Es necesario esperar a la ejecución para que ocurra el enlace dinámico al momento de enviar el mensaje

1.5.2 Mensajes, métodos y method lookup

Cuando un objeto le envía un **mensaje** a otro objeto, este último debe actuar en consecuencia, es decir, realizar un conjunto de acciones que se corresponden con el mensaje. Ese comportamiento (código programado) es el **método** y respeta de algún modo el contrato del mensaje: respeta lo que se debe hacer.

Por ejemplo, cuando le envían el mensaje **volar** a *pepita*, ésta debe *consumir energía*. El comportamiento que tiene *Pepita* para consumir energía es desconocido por quien envía el mensaje pero cumple con lo que debe hacer. Es decir que su energía es menor luego de ejecutar el método correspondiente.

El mensaje representa lo **qué** se debe hacer, mientras las instrucciones del lenguaje que se incluyen en el método representan el **cómo** se hace. A la hora de definir el objeto emisor no se conoce el método asociado al mensaje (en este caso, **volar**). Que el objeto emisor no conozca el método que se ejecuta permite **desacoplar** el *qué* del *cómo*, **encapsulando** en el objeto receptor la tarea de elegir cómo cumplir con la **responsabilidad** asociada al mensaje. *Pepita* puede cambiar la manera en que vuela sin afectar al emisor.

La estrategia que utilizan los lenguajes para resolver dónde está el código de un método al enviar un mensaje recibe el nombre de **method lookup**. En este primer caso de ejemplo se

tiene la situación más simple: el método que corresponde ejecutar a partir de la recepción de un mensaje se busca en el objeto receptor de dicho mensaje.

Esta separación entre el *qué* y el *cómo* a través del *method lookup* toma más importancia cuando se diseña una solución *polimórfica*. En el ejemplo hay dos métodos diferentes *energiaQueAporta*, uno en alpiste y otro en manzana. *Pepita* envía el mensaje al objeto apuntado por la referencia *alimento*. No sabe cuál método se ejecutará. En ese momento a través del *method lookup* wollok selecciona el método *energiaQueAporta* adecuado según el receptor del mensaje.

1.5.3 Polimorfismo y Tipos

Un tipo es un nombre que se usa para denotar una determinada interfaz. En el ejemplo de pepita es capaz de comer *alimentos*. El concepto de *alimento* es, para nuestro sistema, un "tipo", ya que no hay un objeto *alimento*, sino que *alpiste* y *manzana* "son" alimentos, o dicho de otra manera: tienen el tipo *Alimento*.

Un tipo es una abstracción: una idea interesante para resolver el problema. Pero, a diferencia de los objetos, no los programamos explícitamente; estos se plasman en el modelo según cómo programemos nuestros objetos.

En su forma más pura dentro del paradigma de objetos, se define un tipo como un conjunto de mensajes. Si un objeto sabe responder a todos los mensajes de ese conjunto, entonces está teniendo el tipo.

Volviendo a nuestro ejemplo. ¿Cuáles son los mensajes que tienen que entender sí o sí los alimentos? Esa es la pregunta a contestar para detectar cuál es el tipo *Alimento*. A diferencia de lo que nuestra intuición nos dice, no es en los objetos manzana y alpiste dónde hay que buscar la respuesta, sino en el objeto que envía mensajes a los alimentos: *pepita*. Es *pepita* quien usa a los alimentos, y si observamos el método *comer(alimento)*, vemos que un alimento debe contestar un único método: *energiaQueAporta()*. Para tener otro alimento en nuestro sistema, alcanza con construir un nuevo objeto que implemente este mensaje. Otros métodos que pueden aparecer en *alpiste* y *manzana*, como *madurar()*, son propios de cada objeto y no forman parte del tipo *Alimento*.

Podemos asociar fuertemente el concepto de tipo con el de polimorfismo: si dos objetos comparten un tipo, entonces pueden ser usados polimórficamente por un tercer objeto.

1.6 Malas prácticas

Es común que una persona principiante genere soluciones con errores típicos de su impericia. En los siguientes apartados se abordan los problemas más recurrentes.

1.6.1 Mala práctica: if que roba responsabilidad

Alguien acostumbrado a la programación estructurada podría pensar una solución que no aprovecha la ventaja del polimorfismo, ya que en el paradigma que domina no cuenta con esa ventaja. Una solución que se base en *if* para determinar el algoritmo a ejecutar suena natural pero trae problemas de diseño que se evitan al utilizar polimorfismo.

```
object pepita {
  var energia = 100

  method comer(alimento) {
    if (alimento == manzana)
      energia = energia + alimento.base() * alimento.madurez()
    if (alimento == alpiste)
      energia = energia + alimento.peso()
  }
}

object alpiste {
  method peso() {
    return 20
  }
}

object manzana {
  const base = 5
  var madurez = 1
  method madurar() {
    madurez = madurez + 1
  }
  method madurez() {
    return madurez
  }
  method base() {
    return base
  }
}
```

El mayor problema de esta solución es que el objeto pepita queda completamente acoplado a los distintos alimentos: Si se agrega un alimento nuevo, hay que además de agregar el objeto que lo modela, agregar una entrada en los ifs de pepita.

Otro problema es el robo de responsabilidad: el objeto pepita ya no solo se preocupa de modelar a la golondrina pepita, si no también debe encargarse de saber cómo altera a la energía cada uno de los alimentos. Queda entonces algoritmos en pepita que están íntimamente relacionados con conceptos que tienen su propia representación como objetos: manzana y alpiste. Si hay un cambio de requerimiento sobre la manzana, hay que revisar código en el objeto manzana y también en pepita, generando mayor dificultad para mantener el código.

Además, los objetos alpiste y manzana quedan *anémicos*. Un objeto anémico es aquel que guarda información pero no tiene comportamiento que la manipule. Lo que es considerado una falla grave de diseño porque ese comportamiento faltante en el objeto anémico queda en algún otro lado difícil de encontrar. En contraposición, se dice que esta versión de pepita es un *God Object* (objeto dios), porque tiene todo el conocimiento mezclado ahí dentro. Un objeto con demasiadas responsabilidades es un objeto difícil de mantener.

Una cuestión importante del paradigma es que divide un problema complejo en muchos problemas más simples. Un modelo que tiene un *god object* implica que las responsabilidades no han sido divididas, y por lo tanto no se ha reducido la complejidad del problema.

Algunas maneras de detectar que el código anterior debería ser reemplazada por una solución polimórfica:

- Se hace comparación para saber cuál es el objeto. Esto es un error grave, pepita nunca debe saber si habla con una manzana o con un alpiste. Solo debe hablar a través del rol que estos objetos cumplen para ella: sabe que son alimentos y por lo tanto debe enviar mensajes que entienden el tipo *Alimento*.
- Hay dos (o más) algoritmos distintos para cumplir una misma responsabilidad y pero no están cada uno en un objeto distinto. Si hay dos maneras de cumplir la misma responsabilidad (calcular la variación de energía al comer), entonces se requiere dos métodos distintos y un único mensaje.

1.6.2 Mala práctica: distintos mensajes para la misma responsabilidad

Esta es una variante de la anterior: es pepita quien mantiene los dos algoritmos de modificación de energía, con todas las desventajas mencionadas anteriormente sobre objetos anémicos, god object y la necesidad de modificar a pepita ante la aparición de un nuevo alimento.

```
object pepita {  
  var energia = 100  
  
  method comerMananza(alimento) {  
    energia = energia + alimento.base() * alimento.madurez()  
  }  
  method comerAlpiste(alimento) {  
    energia = energia + alimento.peso()  
  }  
}
```

La diferencia es que al tener mensajes distintos para cada tipo de alimento, se genera la sensación de que se ha evitado el if consultando por la identidad del alimento.

Esto podría ser cierto si quien envía el mensaje es el usuario desde el REPL, simplemente elige cuál mensaje enviar. El problema ocurre cuando *pepita* es utilizado por un tercer objeto, por ejemplo *rebeca*. *Rebeca* conoce a *pepita* y se le configura un alimento para utilizar cada vez que quiera darle de comer a su ave. Para la selección del mensaje termina siendo necesario un *if* por la identidad del objeto, o alguna variante más macabra:

```
object rebeca {  
  var ave = pepita  
  var alimento = alpiste  
  
  method alimentar() {  
    if (alimento == alpiste) {  
      ave.comerAlpiste(alimento)  
    }  
    if (alimento == manzana) {  
      ave.comerManzana(alimento)  
    }  
  }  
}
```

Es decir, llevar la idea de mensajes distintos para cada algoritmo que tiene la misma responsabilidad no evita el *if*, simplemente lo mueve hacia el emisor del mensaje. Si en *rebeca* se piensa en una solución similar teniendo *alimentarConManzana* y *alimentarConAlpiste* se movería el *if* a otro objeto previo. No resuelve el problema.

La manera de detectar esta mala práctica es observar que hay distintos mensajes para la misma responsabilidad. El polimorfismo ofrece una solución diferente: una responsabilidad implica un único mensaje. Luego muchos métodos distintos para ese mensaje, cada uno en un objeto polimórfico donde escribir el algoritmo correspondiente. El *if* selector no es necesario, porque el method lookup es quien termina generando la selección de manera natural.

1.6.3 Mala práctica: usar referencias globales en los métodos

Para la persona inexperta podría resultar natural dar una solución del problema inicial (antes que aparezca la manzana) utilizando la referencia global del alpiste

```
object pepita {  
  var energia = 100  
  
  method comer() {  
    energia = energia + alpiste.energiaQueAporta()  
  }  
}
```

```
}  
object alpiste {  
  method energiaQueAporta() {  
    return 20  
  }  
}
```

La mala práctica aquí es utilizar directamente la referencia global en el método en lugar de pensar el rol que cumple el alpiste para pepita y generar una referencia para ese rol, ya sea a través de un parámetro (la solución propuesta en este documento) o un atributo.

Esto genera una dependencia muy fuerte de *pepita* hacia *alpiste*. Cuando surge la necesidad de comer también una manzana, termina siendo necesario refactorizar (cambiar la implementación) de *pepita* para que soporte múltiples alimentos, removiendo el uso de la referencia global. En algunos problemas particulares donde se tiene la seguridad de que no será necesario interactuar con un objeto distinto es aceptable utilizar la referencia global.

En el caso de querer utilizar un atributo, éste sí puede ser inicializado usando la referencia global: `var alimento = alpiste`, porque permite que *alimento* pase a referenciar a otro objeto en otro momento. El problema es el uso de *alpiste* dentro de un método

1.6.4 Mala práctica: utilizar atributos cuando se necesitan parámetros

Si la relación entre los dos objetos tiene sentido para resolver un problema en un momento dado y luego ya no se necesita mantener esa relación, lo correcto es que se trate de un parámetro. En el ejemplo de pepita el alimento es elegido cada vez que se le pide comer. Una mala solución que utiliza el alimento como atributo sería:

```
object pepita {  
  var energia = 100  
  var alimento = alpiste  
  
  method comer() {  
    energia = energia + alimento.energiaQueAporta()  
  }  
  method alimento(_alimento) {  
    alimento = _alimento  
  }  
}
```

Para ver el problema debemos analizar como se usa en el REPL

```
wollock:example> pepita.alimento(alpiste)
```

```
wollok:example> pepita.comer()  
  
wollok:example> pepita.alimento(manzana)  
wollok:example> pepita.comer()
```

Notar que cada vez que se quiere dar de comer, hay que enviar 2 mensajes distintos: 1 responsabilidad, 2 mensajes seguidos que siempre van juntos. Además de ser incómodo, es muy propenso a bugs por olvidar enviar un mensaje previo. Por otro lado trae inconvenientes avanzados que caen fuera del contexto de la materia (programación concurrente)

La manera de detectar el problema es viendo que esos mensajes siempre van de a pares, y también que se está guardando permanentemente una referencia a un objeto que luego de la invocación de comer ya no es necesario.

1.6.5 Mala práctica: utilizar parámetros cuando se necesitan atributos

También es una mala práctica el uso de parámetro cuando se necesita un atributo.

En el caso de *rebeca*, la cuidadora de pepita, el requerimiento es configurar el alimento que va a consumir. se pretende utilizar así:

```
wollok:example> rebeca.ave(pepita)           //configuro el ave 1 vez  
wollok:example> rebeca.alimento(alpiste)     //configuro el alimento  
wollok:example> rebeca.alimentar()           //usa la configuracion  
wollok:example> rebeca.alimentar()           //usa la configuracion otra vez  
wollok:example> rebeca.alimento(manzana)     //cambia la configuracion  
wollok:example> rebeca.alimentar()           //usa la configuracion  
wollok:example> rebeca.alimentar()           //usa la configuracion
```

Una buena solución para poder configurar cuando corresponde y mantener la configuración sería utilizar atributos

```
object rebeca {  
  var ave = pepita  
  var alimento = alpiste  
  
  method alimentar() {  
    ave.comer(alimento)  
  }  
  method ave(_ave) {ave = _ave}  
  method alimento(_alimento) {alimento = _alimento}  
}
```

Una mala solución que usa parámetros en lugar de atributos es:


```
object rebeca {  
  
  method alimentar(ave, alimento) {  
    ave.comer(alimento)  
  }  
}
```

El REPL utilizando la mala solución nos da pista sobre el problema

```
wollok:example> rebeca.alimentar(pepita, alpiste)  
wollok:example> rebeca.alimentar(pepita, alpiste)  
wollok:example> rebeca.alimentar(pepita, manzana)  
wollok:example> rebeca.alimentar(pepita, manzana)
```

El problema es que se ha perdido la configuración, cada vez que *rebeca* debe alimentar se le debe decir a quién está cuidando y que alimento hay que darle. Se le está dejando a quien usa el sistema que recuerde cada vez sobre qué ave se trata y cuál alimento darle. Quedó esa información fuera de nuestro sistema, haciendo complicado el uso del mismo. En esta solución *rebeca* no es útil, solo ha quedado como un pasamano.